



A Multiple Environment Available Path Planning Based on an Improved A* Algorithm

Yu Sun¹ · Qingni Yuan^{1,2,3} · Qingyang Gao¹ · Liangyao Xu¹

Received: 13 March 2024 / Accepted: 13 June 2024
© The Author(s) 2024

Abstract

The objective of the path planning for a mobile robot is to generate a collision-free path from a starting position to a target position, aiming to realize a higher quality of path planning, an improved A* algorithm and a hybrid approach incorporating the dynamic window algorithm have been proposed for robot path planning in various environments in this paper. In global path planning, first, a bidirectional search strategy was introduced into to improve the searching efficiency, and an adaptive heuristic function was designed to reduce redundant search nodes. In the meantime, a filtering function for key path nodes and an enhanced jump point optimization method help to remove redundant nodes in the path, reduce turning angles, greatly shorten the path length, and smooth the path using cubic B-spline curves. Furthermore, in local path planning, the combination of key path nodes and the dynamic window approach (DWA) algorithm is utilized to achieve obstacle avoidance in dynamic environments and adjust the heading angle of the section enables seamless locomotion of the robot. Finally, the simulation experiments and physical experiments on the robot were conducted to validate that the proposed improved algorithm significantly improves the speed of path planning while also reducing the length of the planned path and improve the reliability of the algorithm when compared with other algorithms.

Keywords Mobile robot · Path planning · A* algorithm · Bidirectional search · DWA

Abbreviations

B-A*	Bidirectional search A*
AB-A*	Adaptive heuristic function bidirectional search A*
JAB-A*	Jump point search adaptive heuristic function bidirectional search A*
IAB-A*	Improved jump point search adaptive heuristic function bidirectional search A*
DWA	Dynamic window approach algorithm
ABC	Artificial bee colony algorithm

1 Introduction

With the advancement of technology, reducing manpower and resources, improving transportation efficiency, and adapting to harsh environments have become challenges faced by modern logistics. Mobile robot technology plays a crucial role in the contemporary field of warehouse logistics [1, 2]. The hinge of mobile robots lies in path planning which includes global path planning and local path planning. Among these, global path planning is a static path planning referring to find the best path without colliding with obstacles in a known environmental model [3, 4]. However, the environmental model of local path planning exhibits uncertainty corresponding to practical application environment [5, 6]. Given the differences of environmental model between two path planning, the main algorithms are primarily relied upon manifest diversity [7]. In addressing the path planning problem for robots, numerous scholars have employed a variety of approaches and further refined them, including Dijkstra algorithm [8], RRT algorithm, ant colony algorithm [9] and A* algorithm [10, 11]. However, the current algorithm still suffers from suboptimal path length, extended computational runtime, and inapplicability of the generated

✉ Qingni Yuan
qnyuan@gzu.edu.cn

¹ Key Laboratory of Advanced Manufacturing Technology of the Ministry of Education, Guizhou University, Guiyang, China

² State Key Laboratory of Public Big Data, Guizhou University, Guiyang, China

³ School of Mechanical Engineering, Guizhou University, Guiyang, China

paths for practical robotic operations. There are challenges in local path planning algorithms, such as the tendency to get stuck in local optima and the issue of redundant path length. In order to address the issues in robot path planning, this article will employ the A* algorithm for resolution, and further enhance it.

A fascinating route for improving path planning efficiency, shortening path length and smoothing turning angles is the gist of improvement of A* algorithm and root of safe behavior of mobile robots [12, 13]. In regards to improve the path planning efficiency, Wang et al. [14] introduced a parallel approach in the bidirectional search of the A* algorithm, reducing the number of function calls and improving the algorithm's path planning efficiency, the jump point optimization search strategy in the algorithm, reducing the search for redundant nodes and improving path planning efficiency [12, 13, 15, 16], and introduced a bidirectional alternating search strategy in the A* algorithm, reducing computation time and improving search efficiency. While for reducing path length, Zafar et al. [17] proposed a novel improvement to the A* algorithm based on the local tangent method. In this approach, a small number of selected corners are fed into the A* open list, which significantly shortens the algorithm's convergence time. In terms of smoothing turning angles, Eshtehardian and Khodaygan [18] proposed a method that combines the rapidly exploring random trees (RRT*) algorithm with B-spline curves to smooth the generated paths, avoiding collisions and providing correction. All the works demonstrate that introducing other search strategies, improving the evaluation function or enhancing jump point optimization method could help to optimize A* algorithm and stabilize higher efficiency of path planning.

In addition to perfection of the path planning in a known environmental model, the real-world scenarios must be taken into account [19]. In order to solve the problem of low safety and efficiency of underground mine vehicles, a path planning method for underground mine vehicles based on an improved A star (A*) and fuzzy control DWA is proposed [20]. In other words, there exist necessity to consider the suitability of A* algorithm in local path planning where the planned routes may not conform to the actual robot motion model and real-time obstacle avoidance is not possible in the presence of dynamic obstacles [21]. Integrating with other algorithms is a proven solution, notably, the DWA algorithm possesses good obstacle avoidance capabilities. Moreover, it has been shown that a hybrid dynamic path planning algorithm for FAGVs (flexible automated guided vehicles) based on an improved A* algorithm and an improved DWA algorithm [22] avoided suddenly appearing unknown static and dynamic obstacles. Therefore, the refinement of A* algorithm is not only conducive to enhance the efficiency in global path planning but also to the benefit of and achieving complementarity

with other algorithms and promoting the practical applications in local path planning.

Herein, this research conceives a bidirectional search approach to address the issue of long computation time in the algorithm, and an adaptive adjustment of the heuristic function is introduced into to tackle the problem of divergent and redundant search nodes, which could adjust the heuristic function based on the density of obstacles near the robot. On this basis, in view of convoluted loopback paths, an enhanced jump point optimization technique is proposed to reduce path length. Meanwhile, B-spline curves are applied to smooth out the path, satisfying the actual motion requirements of the robot. In addition, in the face of static obstacle environments, key nodes of the global path are extracted and combined with the DWA algorithm to generate collision-free smooth paths in dynamic obstacle environments. Our work clearly disentangles the intrinsic correlation between refinement of A* algorithm and improved path planning, which is pivotal for ensuring the precise behavior of mobile robots.

1. The primary contributions of this paper are as follows: global path planning algorithm. In order to address the issue of long computation time in the algorithm, a change in search strategy is proposed using a bidirectional search approach. To tackle the problem of divergent and redundant search nodes, an adaptive adjustment of the heuristic function is introduced, which adjusts the heuristic function based on the density of obstacles near the robot.
2. To address the issue of convoluted loopback paths, an enhanced jump point optimization technique is proposed, which significantly reduces the path length. B-spline curves are applied to smooth out the path, satisfying the actual motion requirements of the robot.
3. In the face of static obstacle environments, key nodes of the global path are extracted and combined with the DWA algorithm to generate collision-free smooth paths in dynamic obstacle environments.

The remaining parts of this paper are arranged as follows: Sect. 2 provides a brief introduction to the robot application scenarios and environment models. Section 3 proposes the improved A* algorithm in this paper and provides detailed descriptions of the improvements in two different environments. Section 4 presents simulation experiments and physical experiments of the proposed algorithm, comparing it with other grid map algorithms to validate its effectiveness. Finally, Sect. 5 concludes the paper and outlines future improvements.

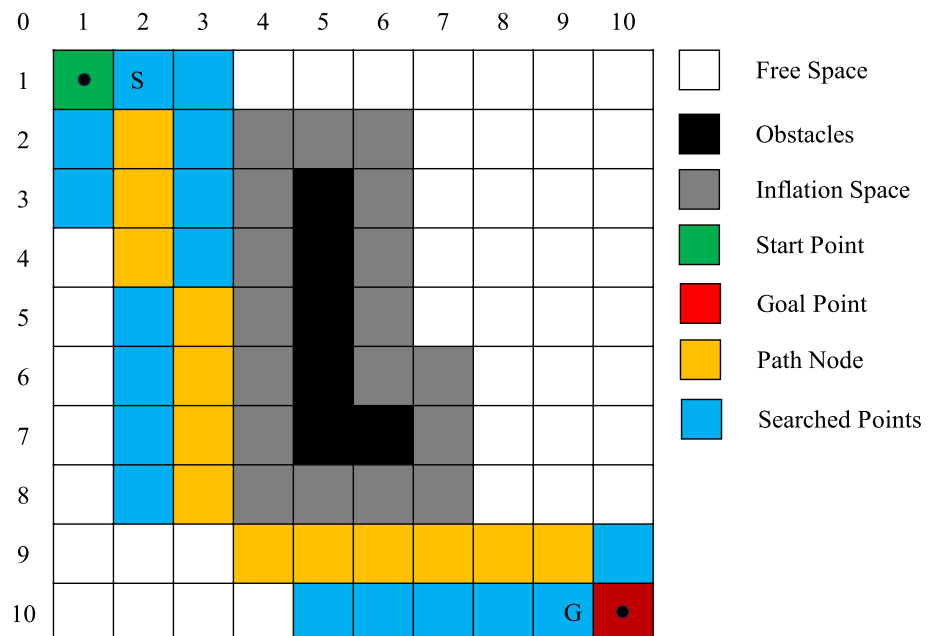
2 Environment Model

The A* algorithm is a typical algorithm based on graph search in global algorithms, and it has a high level of tolerance for processing maps. Whether it is a directed graph or an undirected graph, weighted or unweighted edges, all graph-based search algorithms can effectively handle path planning problems in map environments [23, 24]. In order to accurately depict obstacle information in the environment, utilize grid maps to construct environmental models. These grid maps can offer precise spatial information, with each grid cell capable of representing features such as ground type and obstacle data, ensuring a guarantee of precise depiction of surface characteristics and the distribution of objects [25]. Moreover, the dimensions of the grid are determined by the size of the robot itself, thereby influencing the efficiency and accuracy of the algorithm's search [26]. In this study, in order to enhance the understanding of map semantics by robots, the diverse environmental information on the map has been categorized into distinct state regions as depicted in Fig. 1. Moreover, the robot is considered as a point mass, with each grid being defined as a one-unit size: white squares represent areas that the mobile robot can pass through but has not traversed; black squares symbolize the obstacles that the robot cannot navigate through; gray squares represent buffer zones between the expanded robot and obstacles; green squares represent the starting point; red squares represent the target point; yellow squares represent the path nodes of the robot; blue squares represent the points that the robot has searched.

During the process of path planning using the A* algorithm, the robot is seemed as a point mass and its size is considered as a non-collision distance. After attaching the constraints to obstacles, considering the possibility of the robot scraping or colliding with obstacles in practice, a higher quality and more accurate motion control of the robot would become necessary. In order to address this issue, this paper proposes an optimized approach by adding a buffer zone around the robot's size. Therefore, a layer of points around the obstacles that the robot cannot reach would exist. The robot is depicted as a square grid with each side measuring one unit, and the buffer zone is set to be 1 layer thick. Through this approach, the path should maintain a significant distance from any obstacles to ensure safety, ensuring that collisions would not occur during actual movement.

Ensuring an establishment of an impeccable framework demands a tailored adjustments for the A* algorithm. Generally, the A* algorithm conducts a search of four neighboring nodes during each expansion, and the turning angle is always 90° as depicted in Fig. 2a. However, the efficiency of the robot's motion is significantly compromised, while the turning angle is limited in 90° in practice. In this passage, leveraging the employment of 8-neighborhood connectivity contributes to enhancing the efficiency and performance of path planning. In addition, in Fig. 2b, after augmenting the number of adjacent nodes from four to eight, the turning angle is not only 90° anymore.

Fig. 1 The A* algorithm search in 10×10 map



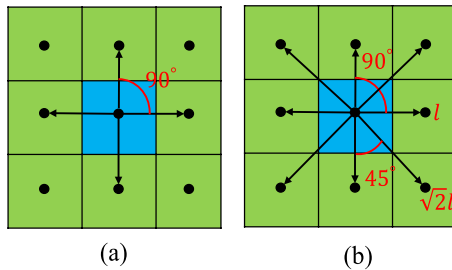


Fig. 2 The connectivity of grid cells in the A* algorithm. **a** Searches 4 neighbors in each expansion. **b** Searches 8 neighbors in each expansion

3 Improved Algorithm

3.1 The Fundamental A* Algorithm

In fundamental A* algorithm, as each node expand, we would proceed to choose the node with the least expansion cost until reach the target point and gain the optimal path [12, 13]. During the search process, the total cost of each node is evaluated by adding up $g(n)$ and $h(n)$. The evaluation function is defined as follows:

$$f(n) = g(n) + h(n) \quad (1)$$

In this context, n denotes the node in question; the evaluation function $f(n)$ signifies the aggregate cost associated with the node at hand, $g(n)$ articulates the cost incurred in traversing from the point of origin to the node under consideration, whereas $f(n)$ is a heuristic function that proffers an estimation of the expected expense to reach the destination from the node currently being evaluated. The evaluation function greatly influences the search process of A* algorithm. Moreover, when it comes to selecting a heuristic function, there exists several available options. Especially, when $f(n) = h(n)$, the A* algorithm would search for optimal path nodes exclusively and thus significantly reduce the computation time. Generally, the heuristic function $h(n)$ encompasses the Euclidean distance function, the Manhattan distance function, and the Chebyshev distance function. In this manuscript, we opt for the Euclidean metric as our heuristic function, which calculates the direct distance between two coordinate points and is deemed to be more representative of actual paths than other functions.

The current node is (x_n, y_n) , the target node is (x_G, y_G) , and the heuristic function relying on the Euclidean distance metric may be articulated in the ensuing manner:

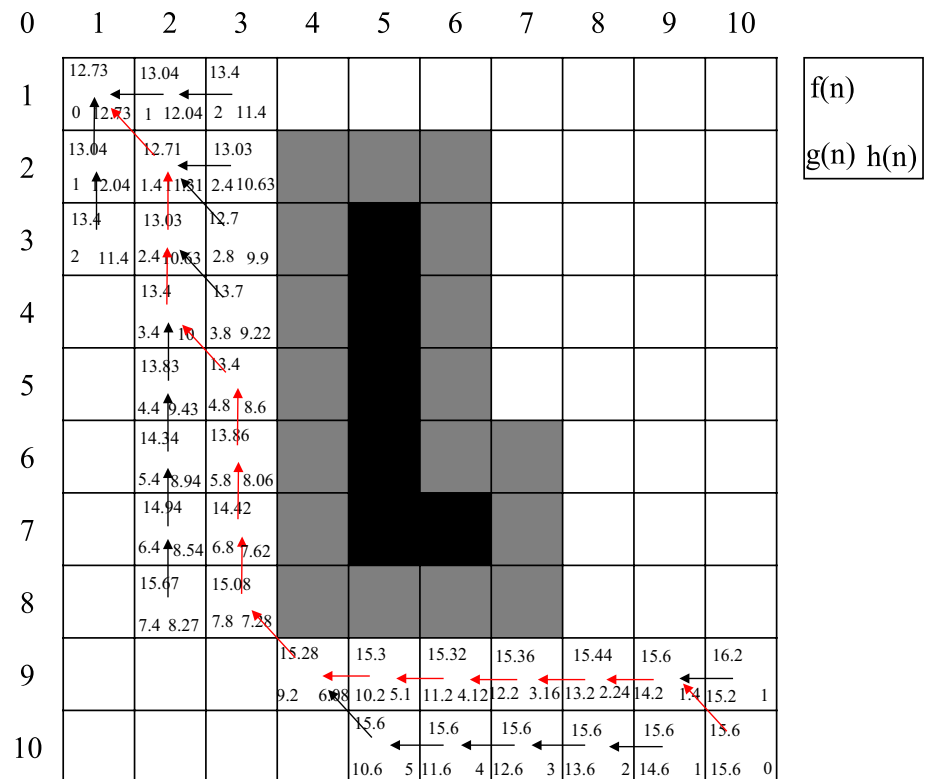
$$h(n) = \sqrt{(x_G - x_n)^2 + (y_G - y_n)^2} \quad (2)$$

In addition, the graphical representation in Fig. 3 clearly portrays the allotted values of g , h , and f for each node as determined by the fundamental A* algorithm. The numerals positioned in the upper-left, inferior-left, and inferior-right vertices of the quadrilaterals in Fig. 3 correspond, respectively, to the functions $f(n)$, $g(n)$, and $h(n)$ as delineated in Eq. (1). Furthermore, at regular intervals, there are arrows pointing to their respective parent nodes, thus orchestrating a unidirectional path from the starting point to the desired destination. The original value of function $g(n)$ at the inception is 0. In addition, $h(n) = 12.73$ is calculated through Eq. (2). Then, the value of $f(n)$ is calculated as 12.73 through Eq. (1). The value of $h(n)$ at the target point is 0, which is computed through an extended accumulation resulting in $g(n) = 15.6$. Subsequently, the value of $f(n)$ is calculated as 15.6 through the equation. In the output of the algorithm, each node only has one parent node. The searching activity is conducted towards on direction, focused on the initial juncture and the ultimate destination. However, during this process, multiple unnecessary nodes are generated, occupying memory and wasting search time. The reason for this occurrence is that within the searched nodes, the difference between the $g(n)$ and $f(n)$ of adjacent nodes is not significant enough to make timely decisions. In addition, due to the algorithm's search pattern of continuous expansion towards neighboring nodes in eight directions, the generated paths could achieve the shortest path according to the algorithm, however, which is evidently not the actual shortest path.

3.2 Enhance the Path Planning Algorithms of A* and Dynamic Window Approach (DWA)

To remediate the challenges associated with protracted computational duration and sluggish retrieval velocity, redundant search nodes, non-smooth paths, and inability in dynamic environments for fundamental A* algorithm, this paper proposes an improved version called IAB-A* which combined A* algorithm with the DWA algorithm. First, the A* algorithm is enhanced using a bidirectional search strategy to reduce computation time. In addition, an adaptive heuristic function is introduced to minimize redundant search nodes. Furthermore, the resulting path is optimized via incorporating jump point optimization, effectively reducing its length. Finally, the trajectory is refined through B-spline interpolation. In the context of local path planning, key path points are extracted and combined with the DWA algorithm to enable obstacle avoidance in dynamic environments.

Fig. 3 The g , h , and f values of path planning by the A* algorithm



3.2.1 Bidirectional Search Strategy

The A* algorithm, depicted in Fig. 4a, is a unidirectional search algorithm which requires significant computation time. To tackle this matter, this manuscript delineates the integration of a bidirectional search methodology into the framework of the A* algorithm. This approach commences the pursuit concurrently from both the origin and the destination, with each path considering the other as its endpoint, resulting in an improved search speed. In addition, the search process of advanced A* algorithm is shown in Fig. 4b. First, given the starting node and the target node, creating two empty open lists for each of them to store the nodes to be explored. Simultaneously, two empty closed

lists are created for the starting node and the target node, respectively, to store the nodes that have been explored. Commencing from the initial node and the designated destination node, the A* algorithm utilizes its heuristic function to meticulously explore the most advantageous trajectory, guided by the principles of its heuristic criteria. During the search process, when selecting which node to be explored, the selected node needs to be checked whether has already appeared in the closed list. If the answer is yes, it indicates the two directions have met, and the shortest path has been found.

The pseudocode for the path planning process based on the improved A* algorithm is as follows:

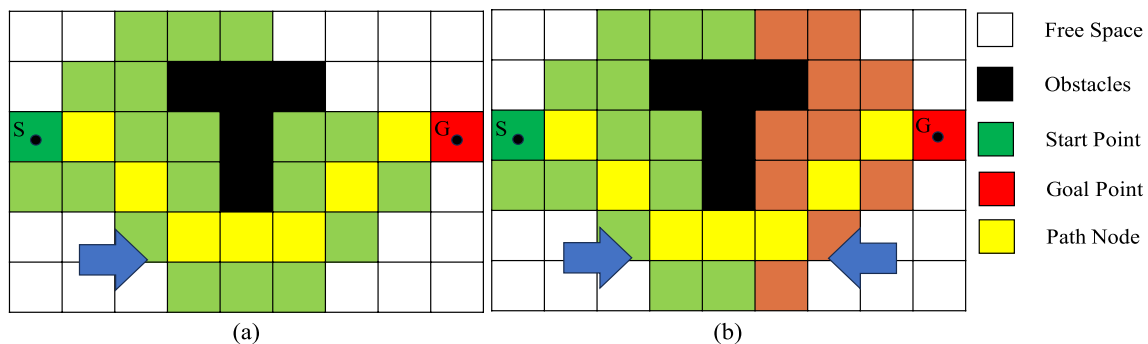


Fig. 4 The search model of algorithms. **a** The search model of A*. **b** The search model of B-A*

Algorithm 1 (The B-A* algorithm)

```

1 Initialize the environment model; Inflate Robot,  $R$  of inflation = 1;
2 Create two empty open lists and close list; add the starting point  $S$  to the FORWARD OPEN and add the starting point  $G$  to the BACKWARD OPEN;
3 While Both FORWARD list and BACKWARD list are not empty do
4 Find the forward current node with the smallest  $f$  value in the FORWARD list and BACKWARD list separately;
5 Move current node to the FORWARD CLOSE and BACKWARD CLOSE separately
6 If there are common nodes between two sets
7 Output the FORWARD PATH and BACKWARD PATH Integrate the FORWARD PATH and BACKWARD PATH into an optimal path
8 else
9 According to the 8 adjacent nodes of NF, update the FORWARD OPEN and the BACKWARD OPEN;
10 End if
11 End while

```

To corroborate the efficacy of the Bidirectional A* algorithm, an experimental simulation was carried out using Python 3.9. In addition, the primary objective revolves around creating paths devoid of collisions across various workspaces, specifically in two 22×22 and 42×42 grid

maps. As shown in Fig. 5, the number of nodes searched by the bidirectional A* algorithm exhibits difference in direction from with that of the A* algorithm. The simulated data for two algorithms are presented in Table 1. while the count of analyzed nodes does exhibit a slight divergence amidst the two algorithms, the bidirectional A* algorithm showcased a reduction in search calculation time by 45–51%. This indicates that the bidirectional A* algorithm is capable of finding paths faster than the A* algorithm, while saving approximately half of the computation time. In addition, as the map's dimensions expand, the bidirectional A* algorithm incrementally converges with the singular A* algorithm with respect to the volume of nodes scrutinized.

Table 1 Conduct experiments on the implementation of the A* algorithm and the bidirectional A* across various maps

Map (size)	Average calculation time/ms		Number of searched nodes	
	A*	B-A*	A*	B-A*
1 (22×22)	38.46 ± 2.29	18.75 ± 1.38	113	132
2 (22×22)	39.29 ± 2.14	17.60 ± 1.35	103	140
3 (42×42)	227.18 ± 5.33	137.66 ± 3.17	653	612
4 (42×42)	135.43 ± 4.41	74.49 ± 3.68	406	416

The reported average computation time values are the averages of 20 executions, with the standard deviation indicated

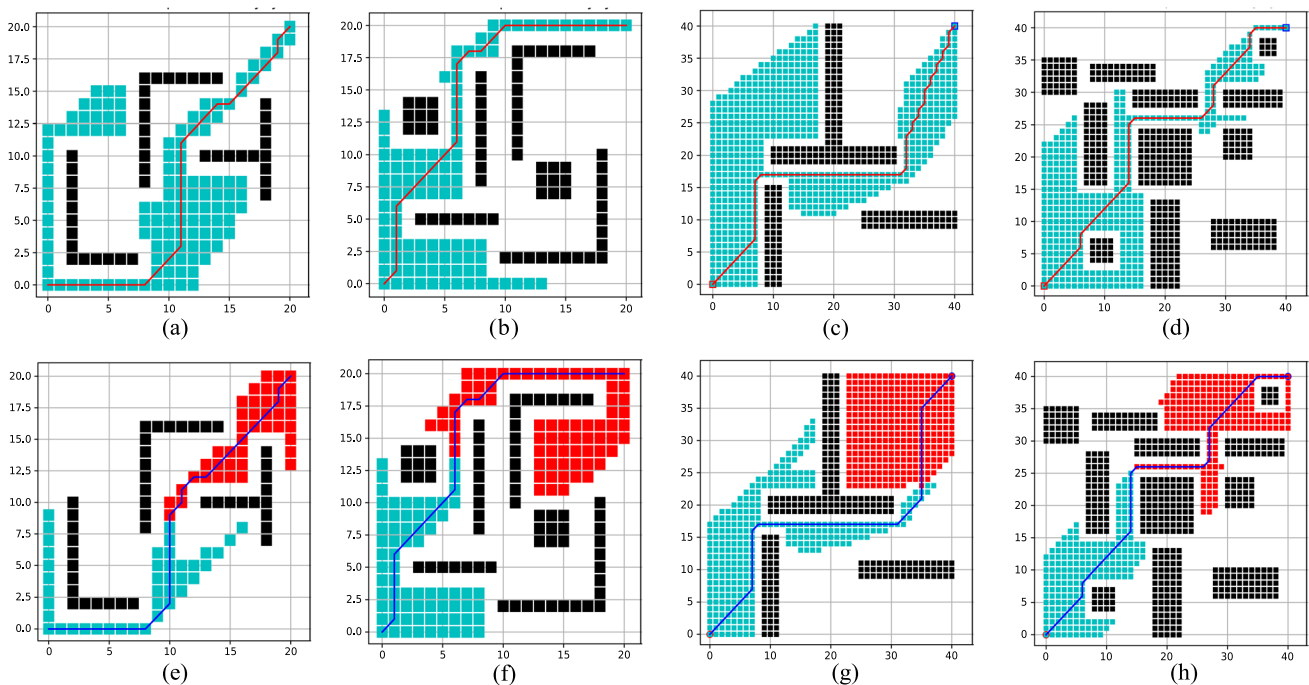


Fig. 5 The simulation results of the A* algorithm and BAS-A* algorithm. **a** Path of A* on map1. **b** Path of A* on map2. **c** Path of A* on map3. **d** Path of A* on map4. **e** Path of B-A* on map1. **f** Path of B-A* on map2. **g** Path of B-A* on map3. **h** Path of B-A* on map4

3.2.2 Adaptive Heuristic Function

As previously noted, the bidirectional A* algorithm typically exhibits superior proficiency in the domain of pathfinding relative to its conventional counterpart. However, when encountering complex obstacle scenarios, the number of search nodes in the bidirectional A* algorithm may surpass that of the A* algorithm due to $f(n) = g(n) + h(n)$. At the beginning of the search, the heuristic value $h(n)$ is considerably large due to the considerable distance from the target point [27]. With a balanced heuristic function, the algorithm tends to prioritize expanding towards the surroundings, conducting a breadth-first search, resulting in numerous irrelevant nodes that waste computational time, as shown in Fig. 5 and the notable surge in computational burden substantially impacts the efficacy of pathfinding. The appraisal function stands as an indispensable element within the ambit of the bidirectional A* algorithm. A pertinent appraisal function not only guarantees the pursuit of the most superior trajectory but also curtails the enumeration of nodes scrutinized. Consequently, this elevates the algorithm's proficiency immeasurably. Therefore, an adjustment for the heuristic function exhibits necessity.

This manuscript elucidates enhancements to the heuristic modality within the bidirectional A* algorithm through the dynamic refinement of the heuristic function. Considering the distance between robot and obstacles, the adjustment of the ratio between $g(n)$ and $h(n)$ in the heuristic function of both the start and target nodes is determined. This approach significantly diminishes the exploration of superfluous nodes, thereby enhancing the efficacy of trajectory formulation. For instance, when $h(n) = 0$, utilizing Dijkstra's algorithm to search every grid until the target node is found ensures the shortest path. However, this approach results in overly redundant node search. Moreover, when the ratio between $g(n)$ and $h(n)$ approaches to 1, the A* algorithm would emphasize a balance between breadth and depth during path planning. When the ratio is greater than 1, the A* algorithm may prefer to slow down the search speed, focusing more on details and node expansion. Conversely, when the ratio is smaller than 1, the A* algorithm may prioritize accelerating the search speed, emphasizing narrowing the distance to the target node [28].

As depicted in Fig. 6, l represents the distance between the nodes in the buffer zone and the nearest obstacle, and d_i represents the distance between any selected path node and its nearest obstacle. Assuming that the distance between the robot and the obstacle is exactly equal to the buffer zone distance l , the ratio between $g(n)$ and $h(n)$ is set to 1. When the separation between the robot and the impediment surpasses the demarcated buffer zone, the distance d_i to the nearest obstacle is calculated, and the ratio between l and d_i serves as the ratio between $g(n)$ and $h(n)$, enabling

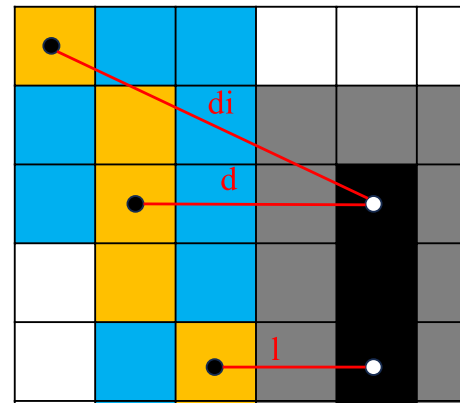


Fig. 6 Adaptive discriminant graph

automatic adjustment of the search speed. However, when the ratio between $g(n)$ and $h(n)$ becomes excessively large, suboptimal search paths may arise. Therefore, this ratio is constrained to be within 3, with a maximum ratio of 3. Consequently, the new heuristic function for the improved A* algorithm is as follows:

$$f(n) = l * g(n) + d_i * h(n) \quad (3)$$

Based on the new heuristic function, as the robot's position moves further away from obstacles, the proportion of $h(n)$ will increase, leading the algorithm to favor a fast search mode. Conversely, as the robot approaches obstacles, the ratio between $h(n)$ and $g(n)$ tends towards equilibrium, prompting the algorithm to favor an expansive search mode.

The simulation results for the bidirectional A* algorithm (B-A*) and the improved bidirectional A* algorithm (AB-A*) on a 22×22 grid and a 42×42 grid is shown in Fig. 7. The AB-A* algorithm, based on an enhanced adaptive heuristic function, has fewer searched nodes compared to the B-A* algorithm, while there is a slight increase in path nodes. According to the data in Table 2, the improved bidirectional A* algorithm reduces the number of searched nodes by approximately 24.4–58.8% compared to the bidirectional A* algorithm. The number of terminal nodes has augmented in comparison to those encountered within the A* algorithm; however, the overarching trajectory of the path remains unaltered. According to the results, we can conclude that the density of obstacles along the way directly affects the number of searched nodes and the speed of the search.

3.2.3 Enhancing Jump Point Filtering Optimization

During the process of searching of A* algorithm, the final formed path is composed of adjacent nodes connecting the

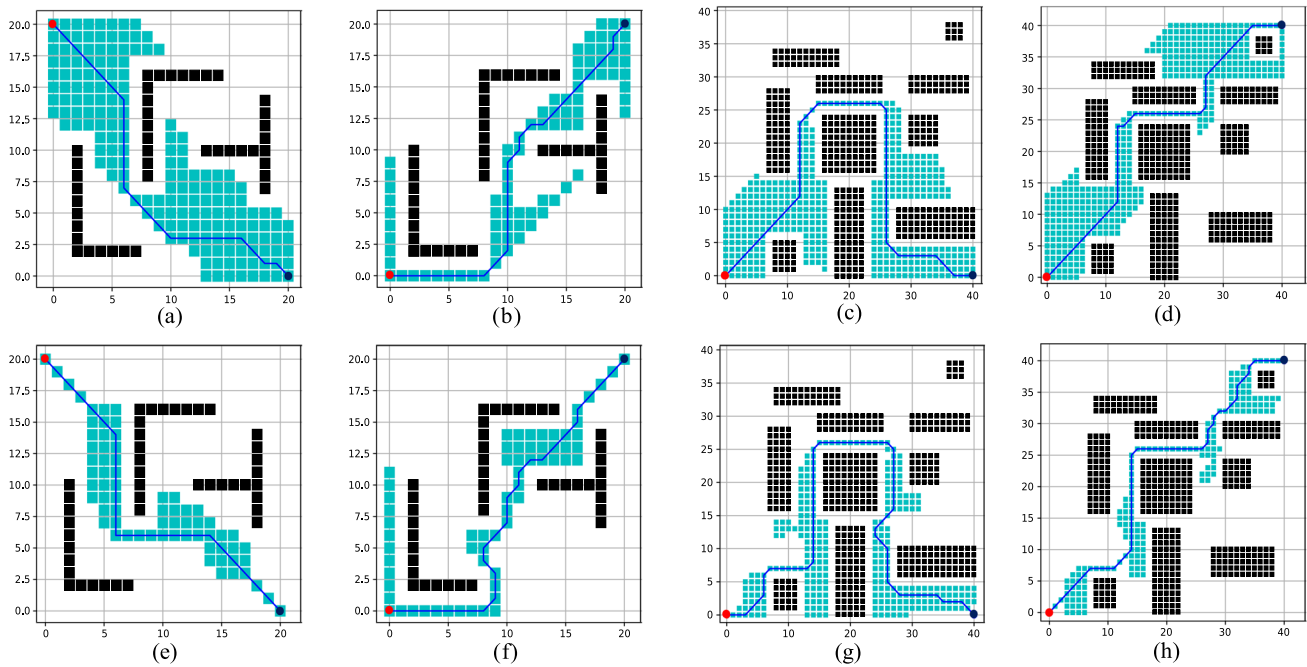


Fig. 7 The simulation results of the B-A* algorithm and AB-A* algorithm. **a** Path of B-A* on map11. **b** Path of B-A* on map12. **c** Path of B-A* on map41. **d** Path of B-A* on map42. **e** Path of AB-A* on

map11. **f** Path of AB-A* on map12. **g** Path of AB-A* on map41. **h** Path of AB-A* on map42

Table 2 Search and path node quantities of bidirectional A* algorithm and enhanced bidirectional A* algorithm in different maps

Map (size)	Number of path nodes		Number of searched nodes	
	B-A*	AB-A*	B-A*	AB-A*
1 (22×22)	29	30	170	70
2 (22×22)	31	31	82	62
3 (42×42)	73	83	440	290
4 (42×42)	59	63	416	172

starting point and the target point. The search pattern of the A* algorithm is limited to eight directions, and each node is expanded and evaluated sequentially, maybe result in convoluted paths which are not the shortest in complex maps or graphs. As shown in Fig. 8, the enhanced bidirectional A* algorithm efficaciously diminishes the duration of path planning and concurrently lessens the quantity of nodes scrutinized. Nevertheless, the distance between nodes becomes greater and the envisaged route continues to encompass numerous superfluous nodes for turning. This component resolves the aforementioned issue by sieving out crucial path nodes via jump point optimization, resulting in the creation of more concise paths in comparison to those charted by the bidirectional A* algorithm [29]. Therefore, the enhanced pathway is more favorable

for regulating the movement of the autonomous robot during real-time navigation.

The core concept of jump point optimization is to skip certain node expansions and evaluations during the search process in order to reduce computational load. By identifying and skipping unnecessary nodes in the search process, the A* algorithm can find the shortest path more quickly. The precise process entails transferring the path nodes derived from the A* algorithm, which connects point A to point B, into a novel set referred to as P . The starting point is labeled as P_0 , and the numbering continues until the destination point, labeled as P_G . Starting from P_0 , each point is connected to the subsequent points, and the distance between the connecting line and the obstacles is compared to the distance of a buffer zone. If the distance between P_0 and P_i is smaller than the buffer distance, then the node P_{i-1} becomes a necessary point and is placed into the set N . Furthermore, P_{i-1} is updated to become P_0 , and the process of connecting the remaining nodes continues until reaching the destination point.

As depicted in Fig. 8a, the verdant enclosure symbolizes the origin, the crimson enclosure signifies the destination, the lavender enclosures correspond to the strategically optimized pivotal nodes along the trajectory, and the aureate enclosures represent the waypoints along the route. The green line represents the path obtained from the A* algorithm, while the red line portrays the path after optimization. Obviously, the optimized path has significantly

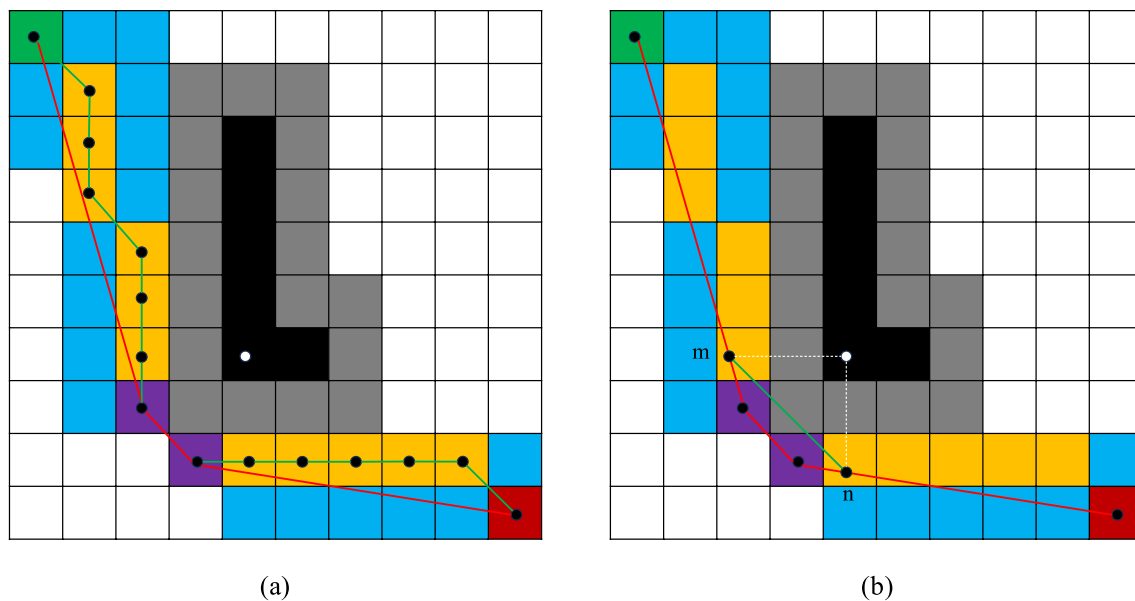


Fig. 8 Improved AB-A* algorithm after path filtering. **a** Jump point search. **b** Improved Jump point search

reduced the distance and minimized the turning angles. However, it should be noted that even after optimization, the path remains affected by the grid-based nature of the A* algorithm, and each node is still based on grid cells and lacks precision. In practice, the locations of robot are not limited to grid points, which calls for further optimization of the path after point selection. As shown in Fig. 8b, the white dots near the obstacles indicate the points which influence the path planning in the segment. Setting one of these points as the origin and drawing a ray intersecting with the planned path at points *m* and *n*, connecting these two points would yield a shorter path, depicted by the green line segment. The new path passes through a buffer zone and maintains a safe distance from the obstacles. In addition, the new path alters small angles in the original path to larger angles, resulting in a shortened path length.

The serpentine trajectory navigating impediments has been reconfigured into a more succinct and conducive route, optimized for robotic control. Reinforced jump points increase the turning angles of the path, thus reducing the path length and make it more in line with the real situation (Fig. 9).

The simulation results and information of the path node after applying jump point bidirectional A* Algorithm (JAB-A*) and improved jump point bidirectional A* algorithm (IAB-A*) on different environment maps are shown in Fig. 10 and Table 3. In Fig. 10, the crimson line showcases the trajectory forged by the JAB-A* algorithm, while the sapphire line exhibits the path charted by the AB-A* algorithm, and the ebony line portrays the course created by the IAB-A* algorithm.

Relative to the bidirectional A* algorithm, the bidirectional A* algorithm enhanced by jump point search ostensibly truncates the trajectory by a margin near 8% while concurrently precipitating a substantial diminution in the frequency of directional changes. Moreover, in juxtaposition with the jump point search when aligned with the bidirectional A* methodology, the IAB-A* algorithm manifests a diminution in the trajectory extent by a margin close to 5%, concurrently effectuating a minimization in the frequency and severity of directional transitions. The findings evidently substantiate that the enhanced trajectory fulfills the criteria for ideal route formulation applicable to autonomous mobile machines.

3.2.4 Smooth B-Spline Path

As a parameter curve, the B-spline curve, characterized by an array of control points and knot vectors, has found wide-ranging applications in practical engineering endeavors. In addition, the B-spline is generated by approximating a set of control points. Assuming we have $P_0, P_1, \dots, P_{n+1}$ control points, which determine the direction and boundaries of the spline curve, the definition of a B-spline curve of degree *k* with *n* + 1 control points is as follows:

$$P(u) = [P_0 P_1 \dots P_n] \begin{bmatrix} B_{0,k}(u) \\ \vdots \\ B_{n,k}(u) \end{bmatrix} = \sum_{i=0}^n P_i B_{i,k}(u) \quad (4)$$

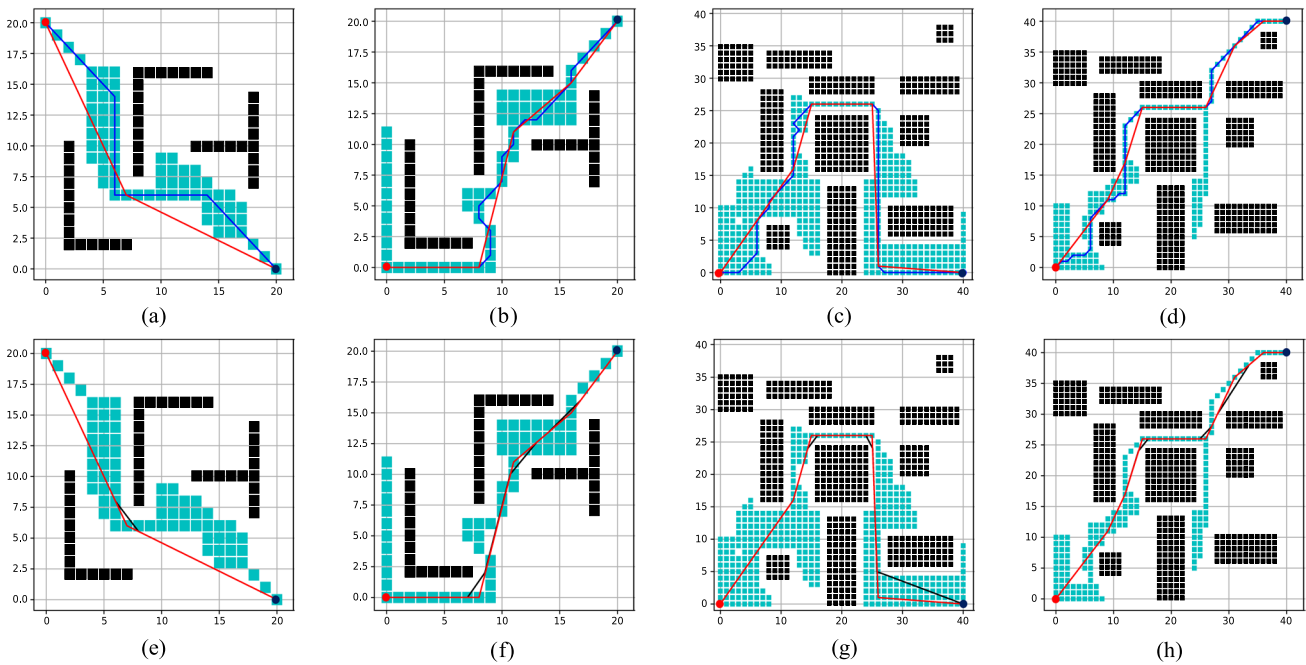


Fig. 9 Path comparison under different scenarios. **a** JAB-A* and AB-A* on map11. **b** JAB-A* and AB-A* on map12. **c** JAB-A* and AB-A* on map41. **d** JAB-A* and AB-A* on map42. **e** IAB-A* and

JAB-A* on map11. **f** IAB-A* and JAB-A* on map12. **g** IAB-A* and JAB-A* on map41. **h** IAB-A* and JAB-A* on map42

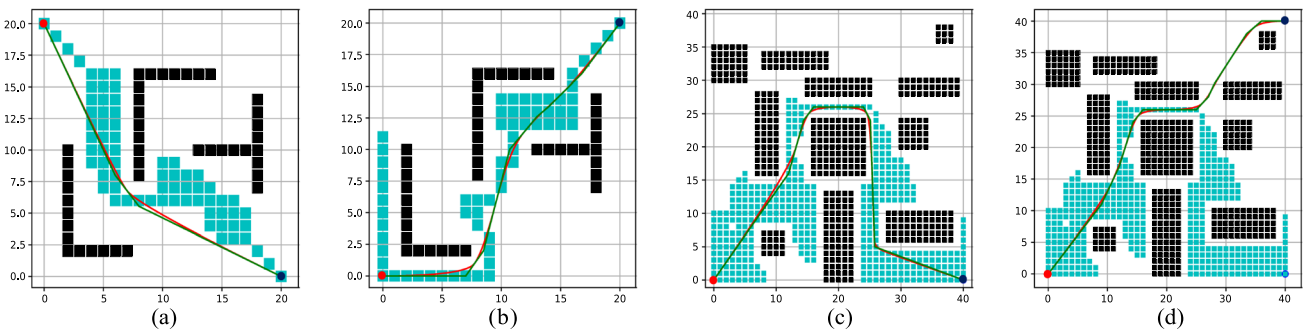


Fig. 10 The smooth path of the improved BAS-A* algorithm after path filtering. **a** Smooth path on map11. **b** Smooth path on map12. **c** Smooth path on map41. **d** Smooth path on map42

Table 3 Comparison of simulated path lengths

Map (size)	Length of path		
	AB-A*	JAB-A*	IAB-A*
1 (22×22)	32.38	29.97	29.8
2 (22×22)	35.31	32.21	31.49
3 (42×42)	65.94	62.99	61.88
4 (42×42)	86.77	79.5	75.26

In the given equation, $B_i, k(u)$ represents the k th i -degree B-spline basis function, where P_i corresponds to the control points. $k > 1$. u acts as the independent variable. The cubic

B-spline curve is the most commonly used B-spline curve order and it could offer greater flexibility and smoothness, allow for the creation of intricate curve shapes. Moreover, higher degree B-spline curves introduce more control points and computational complexity.

In general, it is essential to strike a balance between the requirements for the shape of the curve, computational efficiency, and operability. Quasi-uniform B-spline curve, where all interior knots are equally spaced with a multiplicity of 1, could be a better candidate.

In Fig. 10, the green path represents the filtered path nodes, while the red path represents the smoothed path fitted with B-spline curves. After smoothing the path, a continuous

curvature path is obtained, which is more advantageous for practical control and improves the actual motion efficiency of the mobile robot.

3.2.5 The Integration of the DWA Algorithm for Local Path Planning

In addition to perfection of the path planning in a known environmental model, the real-world scenarios must be taken into account. In the process of practical path planning, there exists two main challenges. First, the intended trajectory and its curvature ought to exhibit seamless continuity, ensuring effortless navigation for the mobile robot. Second, given the existence of dynamic obstacles within the surroundings, it is imperative to employ real-time obstacle avoidance. This manuscript delineates a sophisticated, composite algorithm that amalgamates an enhanced A* methodology with the dynamic window paradigm for trajectory strategizing. The standard DWA algorithm merely specifies the position of the target point; however, when there are numerous obstacles, the path planning algorithm is prone to falling into local optima, leading to an increase in the overall path length. To address this problem, this paper considers the key nodes obtained from the improved A* algorithm's global path as sub-target points for the local path. By decomposing long, complex obstacle environments into multiple simpler paths, the robot is adept at negotiating unforeseen impediments, arriving at the designated coordinates with both safety and efficacy. The flowchart can be seen in Fig. 11

The specific operational procedure is as follows:

1. Extracting the crucial intermediary node $\{P_0, P_1, \dots, P_{n+1}\}$ generated by the improved A* algorithm h to serve as a local target point. Each pair of adjacent nodes would be the starting points for the new paths, dividing the original path into $n-1$ segments of local paths composed of $\{SOG0, SIG1, \dots, SnGn\}$.
2. Employing the DWA, perform path planning for each new trajectory by initializing the velocity space (v, ω) and heading angle θ (referring to the angle between the path starting point and the X-axis). Formulating a two-dimensional velocity space (v, ω) consists of linear velocity and angular velocity.
3. Building a model for robot locomotion

Applying the DWA converts position control of a moving robot into velocity control. Assuming a short time interval for each prediction, and the robot's locomotion model can be described as follows:

$$\begin{cases} x_{t+1} = x_t + v_x \Delta t \cos(\theta_t) - v_y \Delta t \sin(\theta_t) \\ y_{t+1} = y_t + v_x \Delta t \sin(\theta_t) + v_y \Delta t \cos(\theta_t) \\ \theta_{t+1} = \theta_t + \omega \Delta t \end{cases} \quad (5)$$

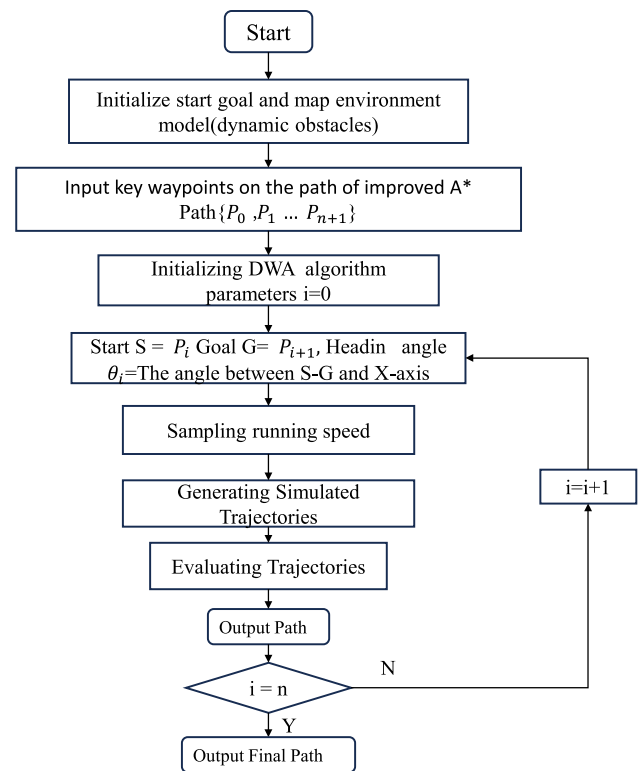


Fig. 11 The flow chart of DWA algorithm

$x_t, x_{t+1}, y_t, y_{t+1}, \theta_t, \theta_{t+1}$ denotes the pose of the mobile robot in the world coordinate system.

4. Constructing a model for the speed of a robotic system:

The DWA leverages continuous positional data for speed sampling, employing a group acceptance sampling plan where the median is used as a quality metric to compute speed [30]. Real-time speed monitoring is achieved through the analysis of changes in positional information.

1. In consideration of the hardware specifications and environmental constraints of the mobile robot, the velocity of the robotic system is subject to certain boundary limitations. We define the set of permissible velocity samples as V_m , within these boundaries:

$$V_m = \left\{ (v, \omega) \mid v \in [V_{\min}, V_{\max}], \omega \in [\omega_{\min}, \omega_{\max}] \right\} \quad (6)$$

The $V_{\min}$ and $V_{\max}$ represent the minimum and maximum linear velocities of the mobile robot, while $\omega_{\min}$ and $\omega_{\max}$ represent the minimum and maximum angular velocities of the mobile robot.

2. The line acceleration and angular acceleration of the mobile robot are both subject to boundary constraints. Assuming that the maximum acceleration

and deceleration magnitudes are equal, the available velocity space for acceleration is as follows:

$$V_d = \left\{ (v, \omega) \left| \begin{array}{l} v \in [V_c - \alpha_{v_{\max}} \cdot \Delta t, V_c - \alpha_{v_{\min}} \cdot \Delta t] \\ \omega \in [\omega_c - \alpha_{\omega_{\max}} \cdot \Delta t, \omega_c - \alpha_{\omega_{\min}} \cdot \Delta t] \end{array} \right. \right\} \quad (7)$$

The V_c and ω_c represent the current linear velocity and angular velocity, respectively, of the mobile robot at the present moment, while $\alpha_{v_{\max}}$ and $\alpha_{\omega_{\max}}$ represent the maximum linear acceleration and maximum angular acceleration of the mobile robot.

3. The dynamic and real-time obstacle avoidance feature in local planning. Taking into account surrounding obstacles, ensuring the robot could be constrained from colliding with obstacles during the process:

$$V_a = \left\{ (v, \omega) \left| \begin{array}{l} v \in [V_{\min} \cdot \sqrt{2 \cdot \text{dist}(v, \omega) \alpha_{v_{\max}}}] \\ \omega \in [\omega_{\min} \cdot \sqrt{2 \cdot \text{dist}(v, \omega) \alpha_{\omega_{\max}}}] \end{array} \right. \right\} \quad (8)$$

$\text{dist}(v, \omega)$ represents the minimum distance between the simulated trajectory and the obstacles under the current velocity.

5. Formulating an evaluation function

Once the velocity constraints for the robot have been determined, it is necessary to evaluate and select the optimal trajectory from multiple sampled trajectories. By comparing the scores based on standard evaluation criteria, the most favorable trajectory can be chosen:

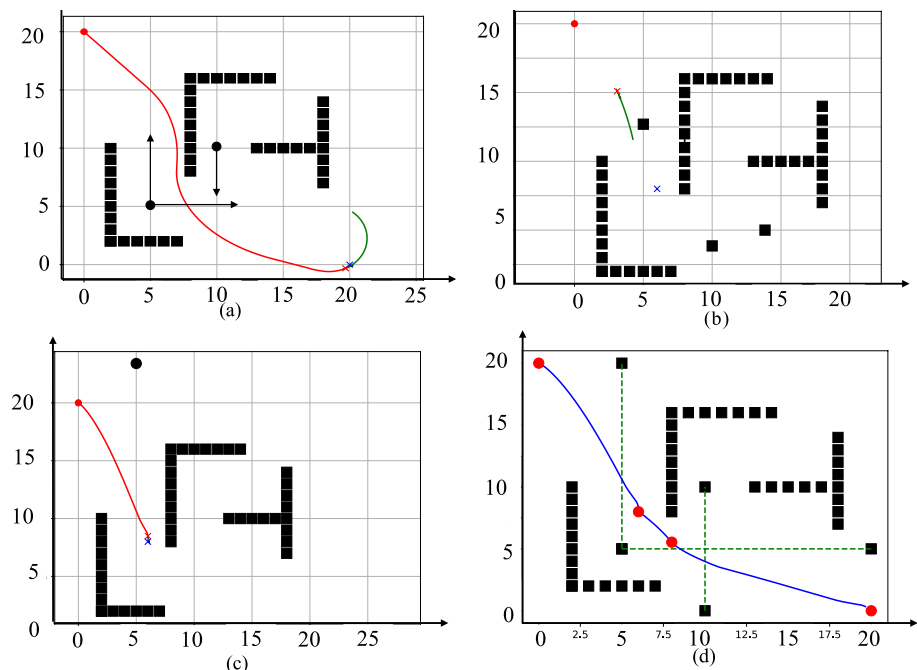
$$G(v, \omega) = \sigma(\alpha \cdot \text{heading}(v, \omega)) + \sigma(\beta \cdot \text{dist}(v, \omega)) + \sigma(\gamma \cdot \text{velocity}(v, \omega)) \quad (9)$$

$\text{heading}(v, \omega)$ serves as an azimuth evaluation function, utilized to assess the error in the angle between the direction of the trajectory endpoint position generated at the current sampling speed and the line connecting it to the target point, referred to as $\Delta\theta$. $\text{dist}(v, \omega)$ represents a distance evaluation function which denotes the minimum distance between the simulated trajectory corresponding to the current velocity and the obstacles. $\text{velocity}(v, \omega)$ is velocity evaluation function, signifying the magnitude of the current speed, wherein α, β, γ are weighting coefficient.

6. Using the dynamic window method to initiate the path planning for a local segment by recalculating the heading angle each time. In addition, repeating this process for each segment to ensuring localized path planning.
7. Integrating each segment of the local path planning trajectories to culminate in the output of a complete and final route.

Verify the fusion algorithm when dynamic obstacles are present. Figure 12a depicts the original DWA algorithm's path planning graph. In Fig. 12b, the individual square obstacles are dynamic, with obstacles 1 and 3 repeating their motion along the y-axis, and obstacle 2 repeating its motion along the x-axis. The quantity of pivotal points, predicated upon the enhanced A* algorithm, has been established at 2. The improved A* algorithm is utilized to extract the key points as local target points for further local path planning.

Fig. 12 The simulation results of the mixed DWA algorithm



The green line represents the predicted path, enabling real-time avoidance of dynamic obstacles. Figure 12c illustrates the schematic diagram of the completed first phase of local path planning. The final path generated after completing the path planning is shown in Fig. 12d.

Although the paths generated by the traditional DWA algorithm are sufficiently smooth, they are noticeably sub-optimal. Therefore, combining the DWA algorithm with an enhanced version of the A* algorithm can achieve a hybrid approach. This hybrid algorithm not only ensures smooth trajectory generation that meets the requirements of robot motion, but also effectively avoids dynamic obstacles and finds the optimal path.

3.3 Algorithm Implementation

Based on the principles of the enhanced IAB-A* algorithm described above, the precise execution methodology of the algorithm is as follows.

1. Establishing an environmental model
2. To commence this endeavor, a fundamental step is to generate two empty open lists, one pertains to the origin node while the other corresponds to the destination node. Subsequently, the commencement node and the objective node are to be inclusively appended to their corresponding open lists.
3. Confirm whether the open list for forward or backward search is empty; if not, proceed to step (4). If it is, this signifies that upon searching through all reachable nodes, if no encounter between forward and backward searches has occurred, the path search is considered unsuccessful.
4. Commence bidirectional search: pursuing the objective node in the forward direction and the initial node in the reverse direction, simultaneously expanding the nodes.
5. The real-time computing of the distance from the robot's current node to the obstacle updates the heuristic function. The cost value of an expanded node is calculated according to formula (5), and neighboring nodes are checked to determine if they are obstacles or already in the closed list. If they are, the node is ignored; if not, the node with the optimal cost value is selected, and the cost and parent node information of that node are updated.
6. When encountering duplicate nodes in the CLOSE list during bidirectional search, it signifies the discovery of a path, and both nodes from the CLOSE lists are added to the collection P .
7. In the set of path nodes, draw connecting lines from the starting point one by one, removing nodes that do not pass through obstacles, keeping the nodes that through obstacles, until reaching the endpoint to obtain key path nodes. Then, enhance jump point optimization based on each key path node and obstacle position, updating the position of key path nodes. Obtain the final path critical nodes. If in a static environment, proceed to step 8; if in a dynamic environment, proceed to step 9.
8. Choose the pivotal waypoints on the essential nodes of the path and designate them as control points for the B-spline curve, in order to achieve a seamlessly refined trajectory.
9. Utilize the points in the set of key nodes along the path as segmented path target points for the DWA algorithm, thereby segmenting the overall path for planning.
10. Initialize the operational parameters of the DWA algorithm.
11. Calculate the initial angle of departure for the robot based on the coordinates of the start and end points of the segmented path.
12. Constructing a model of robotic movement. Determine the sampling rate space. Determine the evaluation function.
13. Select the optimal trajectory.
14. Assemble the segmented paths to create the ultimate trajectory for the robot.

According to the aforementioned steps, the flowchart illustrating the enhanced integration of static and dynamic path planning using IAB-A* algorithm is depicted in Fig. 13:

4 Experimental Evaluation and Analysis

This segment details the empirical validation of the enhanced A* algorithm, juxtaposing its performance against that of the extant Dijkstra, A*, and ant colony optimization algorithms within an identical milieu. The enhanced algorithm's superiority, effectiveness, and reliability are concurrently affirmed. The simulation experiments were conducted in a Python 3.9 environment on a Lenovo computer, with the initial node coordinates set at (0, 0) and the terminal node coordinates set at (20, 20). Scenario 1 was selected as the terrain, and a streamlined, obstacle-free trajectory from the initial point to the terminal point was emulated. In addition, both simulation and physical experiments were conducted using the Baxter robot arm from Rethink Robotics. The principal emphasis of this section resides in a comparative empirical evaluation, scrutinizing the mean duration of search, mean node sampling frequency, and mean trajectory extent, juxtaposed against the performance metrics of the Dijkstra algorithm, bidirectional A* heuristic, and artificial bee colony (ABC) algorithms.

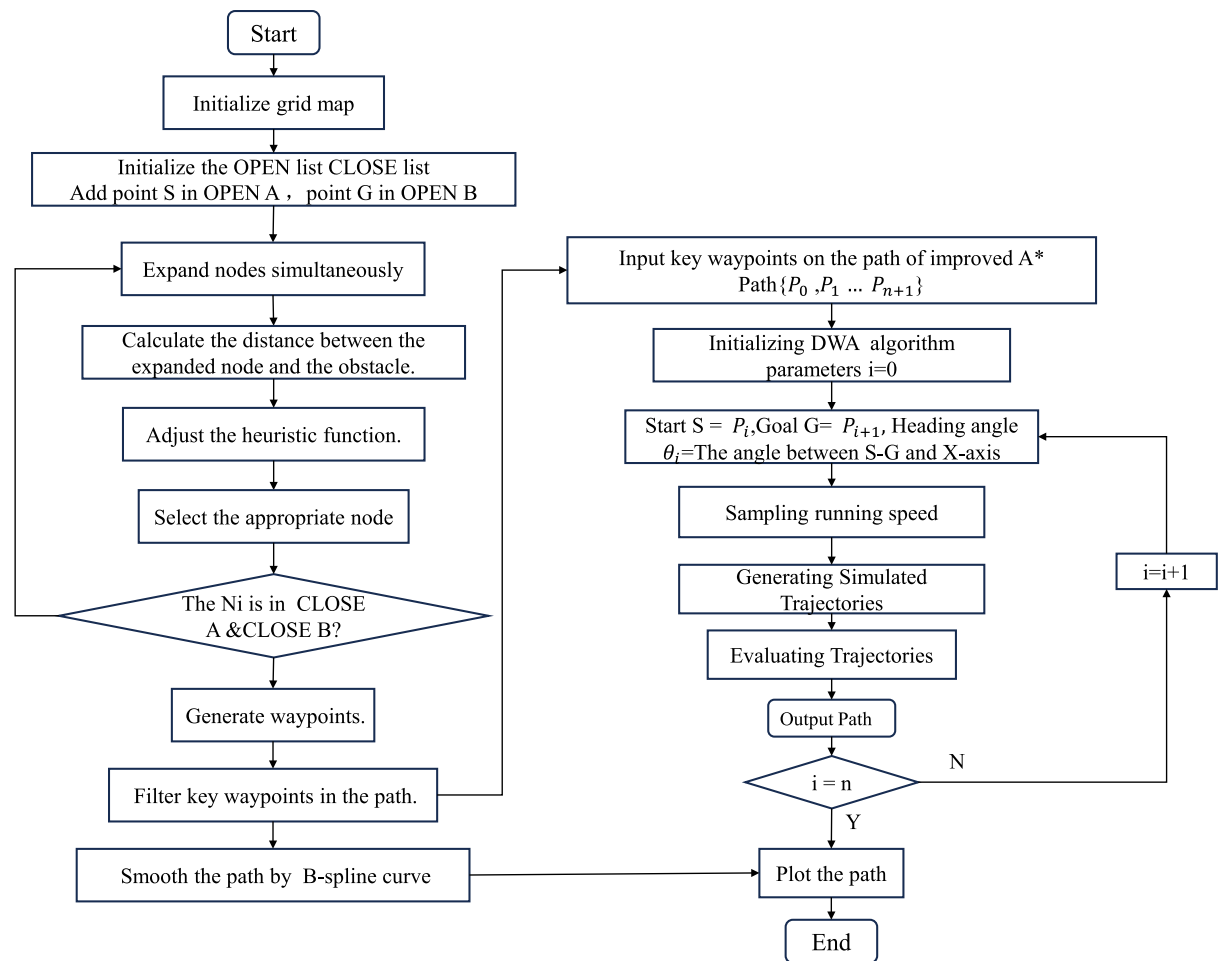


Fig. 13 The flow chart of the mixed A* and DWA algorithm

4.1 Comparative Simulations

In order to verify the preeminence of the algorithm delineated within this manuscript, a comparison was made with Dijkstra, bidirectional A* (A*-B), and ant colony algorithm (ABC) in a stationary environment, as depicted in Fig. 8. Each experiment was conducted 50 times, and the average results are presented in Tables 4 and 5. In Table 4, the path without number of searched nodes are indicated with “–”. The generated path maps and the number of sampled nodes by different algorithms are illustrated in Figs. 15 and 16, respectively.

Based on the data in Tables 4 and 5, the verification was conducted on a smaller map with fewer obstacles (Fig. 14) and larger map with more obstacles (Fig. 15). A comparison was made with other algorithms in terms of algorithm computation time, the number of search nodes, and path length. According to its principle, the Dijkstra algorithm has the highest number of search nodes and longest computation time but can obtain the shortest path. The

Table 4 A comparison of minimalist and obstacle-light maps

Algorithm	Average calculation time/ms	Number of searched nodes	Length of path
Dijkstra	81.76 ± 5.8	296	32.385
B-A*	17.6 ± 1.35	82	35.31
ABC	42.26 ± 1.87	–	40.04
IAB-A*	16.75 ± 1.38	62	31.49

Table 5 A comparison of multi-obstacle maps in a large-scale setting

Algorithm	Average calculation time/ms	Number of searched nodes	Length of path
Dijkstra	224.56 ± 5.8	779	85.94
B-A*	74.49 ± 3.68	416	85.94
ABC	125.78 ± 1.87	–	86.43
IAB-A*	64.49 ± 3.68	172	75.26

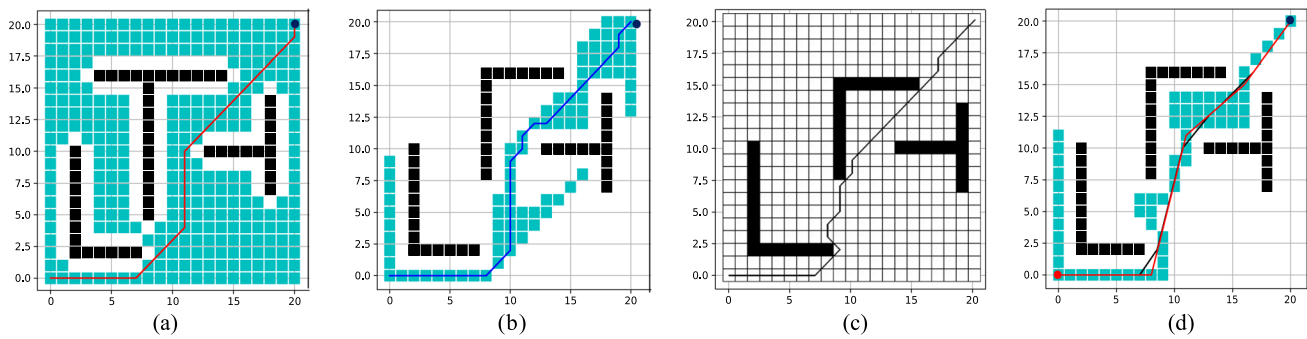


Fig. 14 Different algorithms generate path in Map 1. **a** Dijkstra. **b** B-A*. **c** ABC. **d** IAB-A*

bidirectional A* algorithm (B-A*) and ABC algorithm have intermediate computation times, reduced search node count, and similar overall path directions, albeit with slight differences in local details and only slight variations in path length. In this paper, the proposed IAB-A* algorithm reduces computation time by approximately 40%, decreases the number of search nodes by around 50%, shortens the path length by about 5%, and further reduces the number of turns in the final path. The specific implementation better aligns with the robot motion model. Significant improvements in algorithm efficiency were also observed in maps without obstacles near the starting and target points. The experimental analysis conducted on two maps with different obstacle environments confirms the reliability of the algorithm proposed in this paper.

4.2 Moving in a Real Environment

The efficacy of algorithms within pragmatic settings is impacted by numerous determinants. Consequently, to evaluate the efficacy of the enhanced A* algorithm, we have implemented it for the pathfinding endeavor of the BR280 automaton within an authentic setting, as shown in Fig. 16. The robot utilizes a laser-radar sensing apparatus to accrue data pertaining to the ambient milieu and employs the Cartographer module to ascertain its position and to construct a

bi-dimensional representation based upon the amassed information. As shown in Fig. 17, physical experiments will be conducted in this environment to validate the feasibility of the algorithm. Lastly, the proposed modified IAB-A* algorithm for global path planning will be added as a plugin to the move base package. This module will be used to implement global path planning and local path planning separately, using its global planner and local planner.

Based on the experimental environment configuration depicted in Figs. 18 and 19, two distinct types of obstacles exist in the simulated environment: wall-shaped and pillar-shaped. As a result, two specific experimental scenarios have been established to evaluate both global path planning algorithms and local path planning algorithms.

Figures 18 and 19 illustrate the initial positions of the robot denoted by S and its target location denoted by G , with the robot depicted as a blue square. In Fig. 18, the experimental results of loading the IAB-A* algorithm in a global static environment reveals the ability to plan smooth collision-free paths. Figure 19 presents the path planning process in the presence of additional obstacles, ultimately achieving collision-free paths. The experimental findings demonstrate that the IAB-A* algorithm operates smoothly in conjunction with real-world physical experiments, meeting the motion requirements of actual robots. The experiment

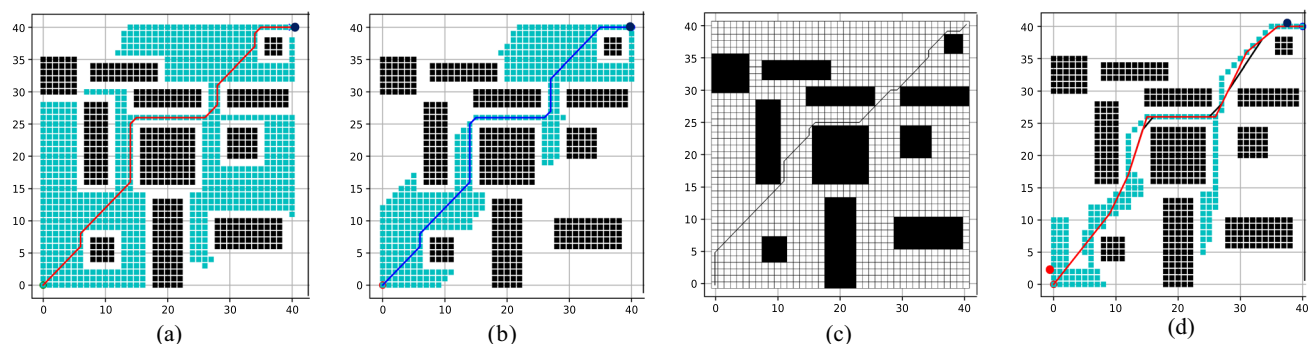


Fig. 15 Different algorithms generate path in Map 2 **a** Dijkstra. **b** B-A*. **c** ABC. **d** IAB-A*

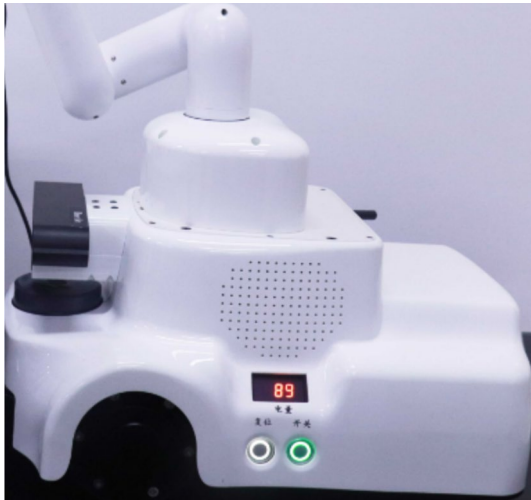


Fig. 16 The BR280 robot

validates the feasibility of the algorithm proposed in this paper, as compared to other algorithms.

In order to validate the feasibility and superiority of the algorithm in practical experiments, experiments were conducted and compared with Dijkstra, bidirectional A*, and ant colony algorithms. Under the same conditions of obstacles, each algorithm underwent 20 simulated experiments, and the results were averaged as shown in Table 6. From the data, it can be observed that the IAB-A* algorithm also exhibits high efficiency in motion planning during the simulated experiments.

Table 6 indicates the average execution time and success rate of four algorithms (Dijkstra, bidirectional A* (A*-B), ant colony algorithm (ABC) and this article algorithm) in practical experiments. The experimental findings demonstrate the smooth operation of the IAB-A* algorithm in conjunction with the physical experiments conducted in actual application scenarios. The algorithm aligns with the motion requirements of real robots, thereby validating the feasibility of the approach proposed in this paper.



Fig. 17 The map of the test scene. **a** Static environment. **b** Dynamic environment

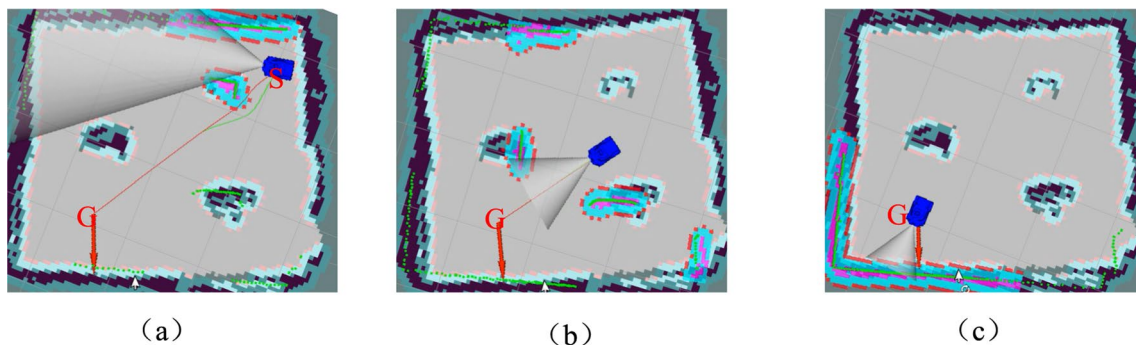


Fig. 18 The performance of the path planning task by the BR280 robot in a real static environment. **a** Plan path. **b** Avoid obstacles robot starting position. **c** Plan path and avoid obstacles

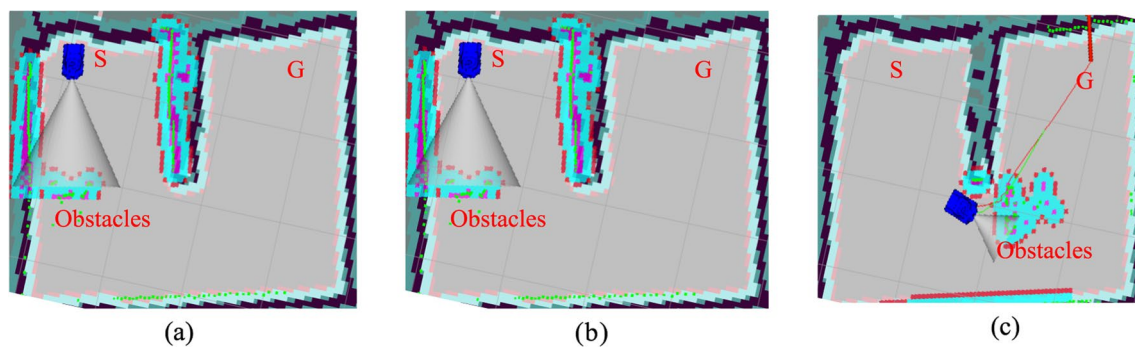


Fig. 19 The performance of the path planning task by the BR280 robot in a real dynamic environment. **a** Experimental environment setup. **b** Robot starting position. **c** Plan path and avoid obstacles

Table 6 Actual performance of various algorithms

Algorithm	Average calculation time/ ms	Rate of successful searches (%)
Dijkstra	12.4	100
B-A*	6.8	100
ABC	9.7	100
IAB-A*	5.4	100

5 Conclusions

This study focuses on the environment of small-scale warehousing logistics, conducting research on path planning for deployed robots. Addressing the issues of global path planning and local path planning, improvements are made based on the A* algorithm to tackle the shortcomings present in path planning with the A* algorithm. We successfully realize instantaneous evasion of impediments and corroborate the practicability of the suggested algorithm through empirical applications. The algorithm under consideration incorporates a novel bidirectional search methodology that alternates directions to augment the efficacy of the path-finding process. Moreover, the construction of an adaptive heuristic function, coupled with a sophisticated enhancement to the jump point optimization technique, successfully mitigates the heightened computational demand resulting from the diminution of superfluous search nodes. Moreover, the reduced turning angle and a smooth path length are obtained via cubic B-spline curves. The final route planning time is reduced by 50%, and the route length is reduced by around 5%. In addition, considering the practical difficulty of acquiring perfect situational awareness during local path planning, integrating with DWA algorithm is a proven solution to generate collision-free smooth paths in dynamic obstacle environments. The empirical outcomes observed

within various grid settings affirm that the algorithm presented herein markedly enhances the rapidity of the navigational route formulation while concurrently abbreviating the extent of the trajectory. Physical experiments validate the feasibility of the proposed algorithm in practical applications and demonstrate notable improvement in execution speed compared to similar grid-based mapping algorithms. The performance of the algorithm is superior to traditional approaches and is suitable for obstacle environments under various conditions, as indicated by the simulation experiments. The advantage of the proposed method in this paper lies in its excellent path quality and real-time performance; however, there is an issue in the local path planning algorithm that integrates the DWA algorithm, which involves long computation time and the inability to handle a path as a whole. Future research tends to lean towards using intelligent algorithms to collectively address both global and local path planning issues. The further work will focus on optimizing the algorithm in collaborative settings involving multiple robots instead of being limited in single robot path planning.

Acknowledgements This research was funded by (the National Natural Science Foundation of China) [Grant Nos. (52065010) and (52165063)], (Department of Science and Technology of Guizhou Province) (Grant Nos. [(2022)G140]; [(2022)K024]; [(2023)G094]; [(2023)G125]], [Laboratory Open Project of Guizhou University] (SYSKF2023-089) [Innovation project of Guiyang Science and Technology Bureau: Development of digital twin system for FAST feed source receiver disassembly and assembly robot.] (2022).

Author Contributions Yu Sun: conceptualization, methodology, software, formal analysis, writing original draft, visualization. Qingni Yuan: served as scientific advisors, critically reviewed the study proposal, methodology, writing review and editing. Qingyang Gao: investigation, formal analysis. Liangyao Xu: investigation, formal analysis.

Funding This work was supported in part by the National Natural Science Foundation of China (Project No.52065010 and No.52165063), Department of Science and Technology of Guizhou Province (Project No. [2022] G140; No. [2022] K024; [2023] G094; [2023] G125), Laboratory Open Project of Guizhou University (SYSKF2023-089).

Availability of Data and Materials No dataset was involved.

Declarations

Conflict of Interest All the authors declare that they have conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Guo, Y., et al.: Research progress of path planning methods for autonomous underwater vehicle. *Math. Probl. Eng.* **2021**, 8847863 (2021). <https://doi.org/10.1155/2021/8847863>
- Hichri, B., et al.: Mobile robots path planning and mobile multi-robots control: a review. *Robotica* **40**(12), 4257–4270 (2022). <https://doi.org/10.1017/S0263574722000893>
- Ait Saadi, A., et al.: UAV path planning using optimization approaches: a survey. *Arch. Comput. Methods Eng.* **29**(6), 4233–4284 (2022). <https://doi.org/10.1007/s11831-022-09742-7>
- Saeed, R.A., et al.: A boundary node method for path planning of mobile robots. *Robot. Auton. Syst.* **123**, 103320 (2020). <https://doi.org/10.1016/j.robot.2019.103320>
- Lyu, D., et al.: Robot path planning by leveraging the graph-encoded Floyd algorithm. *Fut. Gener. Comput. Syst.* **122**, 204–208 (2021). <https://doi.org/10.1016/j.future.2021.03.007>
- Xu, K., Su, R.: Path planning of nanorobot: a review. *Microsyst. Technol.* **28**(11), 2393–2401 (2022). <https://doi.org/10.1007/s00542-022-05373-x>
- Dam, T., et al.: Monte-Carlo robot path planning. *IEEE Robot. Autom. Lett.* **7**(4), 11213–11220 (2022). <https://doi.org/10.1109/LRA.2022.3199674>
- Kurdi, H., et al.: Tide-inspired path planning algorithm for autonomous vehicles. *Remote Sens.* (2021). <https://doi.org/10.3390/rs13224644>
- Miao, C., et al.: Path planning optimization of indoor mobile robot based on adaptive ant colony algorithm. *Comput. Ind. Eng.* **156**, 17230 (2021). <https://doi.org/10.1016/j.cie.2021.107230>
- Cheng, C., et al.: Path planning and obstacle avoidance for AUV: a review. *Ocean Eng.* (2021). <https://doi.org/10.1016/j.oceaneng.2021.109355>
- Thoresen, M., et al.: Path planning for UGVs based on traversability hybrid A*. *IEEE Robot. Autom. Lett.* **6**(2), 1216–1223 (2021). <https://doi.org/10.1109/LRA.2021.3056028>
- Li, C., et al.: Global path planning based on a bidirectional alternating search A* algorithm for mobile robots. *Comput. Ind. Eng.* **168**, 108123 (2022). <https://doi.org/10.1016/j.cie.2022.108123>
- Li, D., et al.: Quality-oriented hybrid path planning based on A* and Q-learning for unmanned aerial vehicle. *IEEE Access* **10**, 7664–7674 (2022). <https://doi.org/10.1109/ACCESS.2021.3139534>
- Wang, H., et al.: The EBS-A* algorithm: an improved A* algorithm for path planning. *PLoS ONE* **17**(2), e0263841 (2022). <https://doi.org/10.1371/journal.pone.0263841>
- Chen, T., et al.: A jump point search improved ant colony hybrid optimization algorithm for path planning of mobile robot. *Int. J. Adv. Rob. Syst.* **19**(5), 17298806221127952 (2022). <https://doi.org/10.1177/17298806221127953>
- Mi, Z., et al.: Path planning of indoor mobile robot based on improved A* algorithm incorporating RRT and JPS. *AIP Adv.* **13**(4), 045313 (2023). <https://doi.org/10.1063/5.0144960>
- Zafar, M.M., et al.: LTA*: local tangent based A* for optimal path planning. *Auton. Robot.* **45**(2), 209–227 (2021). <https://doi.org/10.1007/s10514-020-09956-3>
- Eshtehardian, S.A., Khodaygan, S.: A continuous RRT*-based path planning method for non-holonomic mobile robots using B-spline curves. *J. Ambient. Intell. Humaniz. Comput.* **14**(7), 8693–8702 (2023). <https://doi.org/10.1007/s12652-021-03625-8>
- Xiang, D., et al.: Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot. *Sci. Rep.* **12**(1), 13273 (2022). <https://doi.org/10.1038/s41598-022-17684-0>
- Zhang, C., et al.: A path planning method based on improved A* and fuzzy control dwa of underground mine vehicles. *Appl. Sci.* **14**(7), 3103 (2024)
- Bai, X., et al.: UAV path planning based on improved A* and DWA algorithms. *Int. J. Aerosp. Eng.* **2021**, 4511252 (2021). <https://doi.org/10.1155/2021/4511252>
- Wu, B., et al.: Dynamic path planning for forklift AGV based on smoothing A* and improved DWA hybrid algorithm. *Sensors* **22**, 18 (2022). <https://doi.org/10.3390/s22187079>
- An, D., et al.: Intelligent path planning technologies of underwater vehicles: a review. *J. Intell. Rob. Syst.* **107**(2), 22 (2023). <https://doi.org/10.1007/s10846-022-01794-y>
- Ma, Z., Chen, J.: Adaptive path planning method for UAVs in complex environments. *Int. J. Appl. Earth Observ. Geoinform.* **115**, 103133 (2022). <https://doi.org/10.1016/j.jag.2022.103133>
- Chaari, I., et al.: Design and performance analysis of global path planning techniques for autonomous mobile robots in grid environments. *Int. J. Adv. Robot. Syst.* (2017). <https://doi.org/10.1177/1729881416663663>
- Han, B., et al.: Grid-optimized UAV indoor path planning algorithms in a complex environment. *Int. J. Appl. Earth Observ. Geoinform.* **111**, 102857 (2022). <https://doi.org/10.1016/j.jag.2022.102857>
- Sang, H., et al.: The hybrid path planning algorithm based on improved A* and artificial potential field for unmanned surface vehicle formations. *Ocean Eng.* **223**, 108709 (2021). <https://doi.org/10.1016/j.oceaneng.2021.108709>
- Tang, Q., Dou, W.: An effective method for computing the least-cost path using a multi-resolution raster cost surface model. *ISPRS Int. J. Geoinform.* **12**, 7 (2023). <https://doi.org/10.3390/ijgi12070287>
- Niewola, A., Podsedkowski, L.: L* algorithm—a linear computational complexity graph searching algorithm for path planning. *J. Intell. Rob. Syst.* **91**(3), 425–444 (2018). <https://doi.org/10.1007/s10846-017-0748-6>
- Ameeq, M., et al.: A group acceptance sampling plan truncated life test for alpha power transformation inverted perks distribution based on quality control reliability. *Cogent Eng.* (2023). <https://doi.org/10.1080/23311916.2023.2224137>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.